Python project report.docx

by Moupiyali Banerjee

Submission date: 28-Apr-2026 12:20PM (UTC+0530)
Submission ID: 2942228793
File name: Python_project_report.docx (409.42K)
Word count: 3505
Character count: 19782

“Simple Stopwatch”

A

Project Report

submitted in partial fulfillment of the

requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

by

Name :	**Sap Id :**
Moupiyali Banerjee	**590027857**

under the guidance of

Dr. Anushree Shah

School of Computer Science

University of Petroleum & Energy Studies

Bidholi, Via Prem Nagar, Dehradun, Uttarakhand

April – 2026

CANDIDATE'S DECLARATION

I hereby certify that the project work entitled "**Simple Stopwatch using Python**" in partial fulfilment of the requirements for the award of the Degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING and submitted to the Department of Systemics, School of Computer Science, University of Petroleum & Energy Studies, Dehradun, is an authentic record of my work carried out during a period from **March 2026** to **April 2026** under the supervision of **Dr. Anushree Shah, Assistant Professor**

The matter presented in this project has not been submitted by me for the award of any other degree of this or any other University.

Moupiyali Banerjee
590027857

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date: 24.04.2026 **Dr. Anushree Shah**

ACKNOWLEDGEMENT

We wish to express our deep gratitude to our guide **Dr. Anushree Shah**, for all advice, encouragement and constant support she has given us throughout our project work. This work would not have been possible without her support and valuable suggestions.

We sincerely thank our respected Head of the Department, School of Computer Science, University of Petroleum & Energy Studies (UPES), for their constant encouragement and support in completing our project in the Department of Computer Science and Engineering.

We are also grateful to the Dean, School of Computer Science (SoCS), UPES, for providing us with the necessary infrastructure and facilities to successfully carry out our project work. We would also like to express our sincere thanks to our Course Coordinator and Activity Coordinator for their continuous guidance, timely support, and valuable inputs throughout the project.

We would like to thank all our friends for their help and constructive criticism during our project work. Finally, we have no words to express our sincere gratitude to our parents who have shown us this world and for every support they have given us.

Name -
Moupiyali Banerjee

Sap Id:
590027857

ABSTRACT

The project describes the design, implementation and assessment of a fully functional stopwatch created using Python and the Tkinter Library, which is a GUI (Graphical User Interface) toolkit used for creating cross-platform desktop applications. The stopwatch application has a very user-friendly GUI that allows users to easily start/stop/reset the stopwatch and enter lap times.

Some advanced capabilities of the stopwatch application include: Computation and display of current lap times, best lap time, and worst lap time while timing. Therefore, the user is able to receive immediate feedback about their performance while timing. Data can also be persisted by using an Export feature to save session log information (timestamps, lap times and summary statistics) into a formatted text file for future analysis, review and to integrate with other tools.

When developing the stopwatch application, the Python programming language was used, specifically utilizing core programming paradigms of developing objects (OOP-object oriented programming), developing event driven programs (responsive to user actions), and time management for accurate timing down to the millisecond using the time module. The Tkinter components (widgets) used in designing the user-interface (the User will see the screen and interact with the stopwatch) are Label, Button, List box and Frame. With the use of the after() method for managing dynamic updates to the user interface, the stopwatch will display current lap time, best lap time and worst lap time to the user without blocking the main event loop.
With this prototype you will learn how to create a GUI (Graphic User Interface) Application in Python, focusing on different aspects of the application such as Event Handling, State Management, Visualization of Data, and File Input/Output Operations (I/O). This will allow you to gain experience with best practices for usability, error handling, and extensibility for new to intermediate developers. The GUI application is lightweight, portable, and can be extended further with database integration or multi-threaded capabilities to run multiple timers at one time for use in athletic training, testing in labs, or help track a person's productivity for engineers or scientists using their workflow.

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Abstract	i
2	Introduction	1
3	System Analysis	2
4	Design	5
5	System Designs	7
6	Flowchart	8
7	Algorithm	9
8	Implementation	10
9	Output	11
10	Limitations	12
11	Future Enhancement	13
12	Conclusion	14
13	References	15

INTRODUCTION

Stopwatches are precision timekeeping devices that measure elapsed time between two different events. They typically have a start, a stop, and a reset button for accurate timing. Originally there were mechanical or analog stop watches, but most present-day stopwatches are digital and can measure down to the millisecond and may include additional features like lap timing and can be found in many different fields such as in athletics (running, swimming), in science (measuring the rate of chemical reactions), in industry (timing for quality control) and in everyday use (cooking, exercise, or tracking).

Stopwatches give the operator an objective way to measure performance and the length of time spent on any particular activity and create a data-driven decision-making environment. Coaches use stopwatches to analyze lap splits in the training of athletes, researchers rely on stopwatches to create a reproducible environment for experiments, and engineers use stopwatches to accurately benchmark their prototypes and processes.

While there are still reliable hardware stopwatches being used, software stopwatches have advantages over hardware stopwatches by offering more flexibility, lower cost, and more capabilities. Unlike hardware stopwatches, software stopwatches have no moving parts that can wear out—they can run as many timers as desired (allowing for multiple simultaneous timers), and they may provide additional features that are more advanced than those found in hardware timers (such as automatic lap calculations, lap statistics (including best/worst lap times), data exporting (to a file or database), and custom interfaces).

The goal of this project is to design and implement an advanced software stopwatch application using the Tkinter library (in Python) by implementing a GUI that is user-friendly (has easy-to-use controls and displays immediate feedback) so users can see their activity in relation to the elapsed time of each activity. The application will have basic timers; lap recording functionality; display the best and worst lap times, and allow for exporting of lap data as text files (thereby addressing some limitations of commercially available hardware stopwatches). The proposed application will use event-driven programming techniques and the time module in Python to provide accurate tracking of the elapsed time for a user's activity while demonstrating to students in schools such as UPES (University of Petroleum and Energy Studies) how to implement GUIs in practical applications.

System Analysis

2.1 The Current System

Mechanical timepieces and handheld timers are two examples of traditional timers that are fundamentally accurate but have limited functionality, scalability, and data storage capabilities. Theses simple devices typically can perform basic functions (i.e., start, stop, reset, and lap timer) but they offer little in terms of advanced analytical capabilities.

Limitations of Current Systems:

No Data Preservation: Times from laps cannot be preserved; users must jot down times on paper resulting in potential errors and loss of data.

Limited Analysis: No automated computation for metrics such as fastest lap/worst lap, averages, or overall session summary.

Bottlenecks in Scalability: Only one stopwatch can be used at any given time; thus, having multiple concurrent events/session with one timer is not feasible when using traditional timers nor can traditional timers integrate with other software or tools.

Hardware: fixed display size, batteries required, easily damaged, limited personalization.

Functionality Issues: Buttons tend to be small and allow for accidental activation during fast-paced events (i.e. athletic events) and there is no visual graphic indication to user that event is continuing to run in real time. For example, the Timex Ironman sports stopwatch has an example of a stopwatch that displays the lap split by displaying in chronological order but there are no facilities to record elapsed times for racers, so a coach using the Timex Ironman to keep track of an athlete's time over multip'le events will find this method of time-recovery to be terribly inefficient.

The purpose of this proposal is to create a software-based application built using Python 3.x and Tkinter (the native cross-platform GUI framework included with Python 3) to address the issues noted above. The application will be available on Windows, macOS, and Linux operating systems and is designed to provide timing functionality that is both up-to-date and built for precision, ease of use, and extensibility.

The core features of this proposed system are as follows:

User-friendly interface: The application will have large, color-coded buttons for starting/stopping/resetting/laping, along with a visible digital clock that will update in real-time (accurate to milliseconds);

Lap management: The application will allow you to record lap times with one click; all recorded lap times will be displayed in a scrolling list box with their corresponding times;

Performance analytics: The application will automatically calculate and indicate the best (minimum) and worst (maximum) lap times and display the results quickly

Data transport capabilities: The application will feature a Save button that will allow you to create an organized text file (.txt) containing the complete log of your session, along with the lap breakdowns and statistics, for use in archiving, sharing with others, or importing into other applications such as Excel;

Event-driven programming: The application will have an event-driven approach, so that you can use Tkinter's after() method to make the timer and lap time updates occur without stopping anything else from occurring (this will ensure the timer refreshes every 10 ms)

```
+-------------------+    +-------------------+    +-------------------+
|  User Interface  |<--->|  Event Handler   |<--->|  Time Manager    |
| (Tkinter Widgets) |    | (Button Callbacks)|    | (time.time() )    |
+-------------------+    +-------------------+    +-------------------+
        |                    |                    |
        v                    v                    v
+-------------------+    +-------------------+    +-------------------+
| Lap Recorder &    |    | Data Analyzer     |    | File I/O Module   |
| Display (Listbox) |    | (Min/Max Calc)    |    | (Save Session)    |
+-------------------+    +-------------------+    +-------------------+
```

Advantages Over Existing Systems:

Aspect	Existing (Hardware)	Proposed (Software)
Data Storage	None	File export (.txt)
Performance Metrics	Manual calculation	Automatic (min/max)
Platform Dependency	Device-specific	Cross-platform
Cost	₹500-2000	Free (open-source)
Extensibility	None	Easy code mods

This new proposal addresses existing limitations of traditional stopwatches and implements best engineering practices for GUI creation, processing in real time and development centered on user requirements. Therefore, it is perfect for projects undertaken in academia, sport training and experimental timing at UPES.

2.3 Feasibility Study

This project demonstrates the following types of feasibility:

1. Technical: Python/Tkinter is a readily available technology.
2. Economic: There are no hardware costs to develop.
3. Operational: The intended users (non-technical) will find this application easy and intuitive to use.

The estimated timeline to develop a minimum viable product (MVP) for this project is approximately 2-3 weeks.
The platform is developed using the following features; lightweight Standards Python Libraries that help make the platform portable, easy to code, and minimal dependencies.

Three point One- Tools and Technologies
Programming Languages
• Python (Version 3). Python provides an Object-Oriented Programming (OOP) environment, Exception Handling, and precise Time Functionality.
• Tkinter, is the standard Library that can be utilized with Python to create cross-platform Desktop Software. Tkinter provides a wide variety of Widget types such as buttons, labels, and Listbox as well as allowing for Event-Driven Programming.
• datetime module will provide a function datedate.now() which can be used to generate accurate Timestamp when recording laps or logging sessions.

In addition, the
• time module will allow for High Precision Time Measurement using time.time() which has a Millisecond Level of Precision.
• Development Environment is Visual Studio Code- (Preferred IDE) tested to run on Windows/macOS- with UPES laboratory compatibility.

• No outside packages are being utilized within this project, as all functionality was developed using the Standard Library. Consequently, the platform is ready for immediate deployment.

3 System Design
The application uses a single-window MVC architectural method (model view controller) that has a clean, user-friendly interface designed for speedy access to information during operational periods.

GUI Layout Overview:

```
┌──────────────────────────────────────────────────────────────────────┐
│              STOPWATCH v1.0                │
├──────────────────────────────────────────────────────────────────────┤
│ 00:00:00.000        Best: 12.345s    Worst: 15.678s    │
│                Laps (0)                  │
│ ──────────────────────────────────────────────────────────────────────
│                                          │
│ [START]   [STOP]   [RESET]   [LAP]   [SAVE]        │
└──────────────────────────────────────────────────────────────────────┘
```

Button Functions (Workflow) for the User:

START Button - Green in Colour and Large
Function: Start Timer by using start_time = time.time()
Visual Feedback: The Button changes to gray; The Timer Display will start increasing by every 10ms via after(10, update_timer)

Purpose: Starts Timing Session

STOP Button - Red in Colour and Large

Function: Pause Timer and Calculate Elapsed Time: elapsed = time.time() - start_time

Visual Feedback: Timer stops; Button will no longer say STOP but instead say RESUME

Purpose: Stop Timing to Take Break or Review Lap Times

RESET Button - Gray

Function: Removes Everything - Timer Changes to 00:00:00.000, Clears Lap List, and Resets Best/Worst Stats

Visual Feedback: Timer Display = 00:00:00.000 and Lap Count = 0

Purpose: Prepare for New Timing Session

LAP Button - Blue and will only be active when Timer is Running

Function: Mark The Current Elapsed Time as a Lap Time and Add To The Listbox with the Timestamp via datetime.now()

Visual Feedback: New Lap Displays on the List; Best/Worst Updates Automatically if Best/Worst has been Broken
Purpose: Record Split Times (i.e. 400m times when running a race)
SAVE Button - Disk Icon

Function: Save Session Data to "stopwatch_session.txt" As Follows:

Session: 2026-04-23 22:12:00
Total Time: 02:30:45.123
Laps: 5
Lap 1: 12.345s [2026-04-23 22:12:15]
...
Best Lap: 11.890s (Lap 3)
Worst Lap: 15.678s (Lap 2)

- Visual Feedback: Status message "Session saved successfully!"

- Purpose: Data persistence for analysis

Design Principles Applied:

- Progressive Disclosure: Core controls always visible; analytics secondary
- Color Coding: Green=start, Red=stop, Blue=record (intuitive affordances)
- Real-time Updates: 10ms refresh rate prevents lag during fast timing
- Error Prevention: Buttons disable in invalid states (e.g., LAP inactive when stopped)

Simple Drawing :

```
+-------------------+
|     USER          |
| (Clicks Buttons)  |
+---------+---------+
          |
          v
+-------------------+
|   GUI (Tkinter)   |
| Buttons & Display |
+---------+---------+
          |
          v
+-------------------+
|  LOGIC PROCESSING |
| (Timer, Lap Calc) |
+---------+---------+
          |
          v
+-------------------+
|   DATA STORAGE    |
| (Lap Times File)  |
+-------------------+
```

Flowcharts

Flowchart Steps

Start
↓
Initialize variables (counter = 0, running = False)
↓
Display GUI window
↓
User clicks button?
↓
➡ If Start → Start timer → Update time every second
↓
➡ If Stop → Pause timer
↓
➡ If Reset → Reset time to 00:00:00
↓
➡ If Lap → Record lap time
↓
➡ If Save → Save lap data to file
↓
Repeat until user exits
↓
End

ALGORITHM

Start the program

1. Initialize counter = 0 and running = False
2. Create GUI window using Tkinter
3. Display time label on screen
4. When user clicks Start:
    - Set running = True
    - Begin timer loop
    - Increase counter every second
5. When user clicks Stop:
    - Set running = False
6. When user clicks Reset:
    - Set counter = 0
    - Clear lap list
7. When user clicks Lap:
    - Store current time in list
    - Display lap time
8. When user clicks Save:
    - Save lap data to text file
9. Repeat until user exits
10. End program

IMPLEMENTATION

The Stopwatch App is a single Python program with a user interface developed using Tkinter, is time managed by Python's time module, and time stamped using Python's datetime module. The basic functionality of the app is managed by an instance of the Stopwatch class which contains five buttons; START begins the timer and updates the timer every 50 ms using root.after(50, update_timer) in the format of MM:SS.mmm; STOP pauses the timer; RESET clears all data; LAP records the elapsed time in a Listbox and automatically calculates the elapsed best/worst time by comparing the elapsed time against the minimum and maximum running elapsed time; SAVE saves the session data to a .txt file with a date and time stamp that includes the lap times and statistics.

4.2 Explanation of Key Features (Simple Terms)

real-time timer updates (update_timer() method)
Updates the display every 50 milliseconds, using root.after(50, self.update_timer).
time elapsed is calculated as time.time() - start_time.
time is formated to be very readable: MM:SS.mmm.

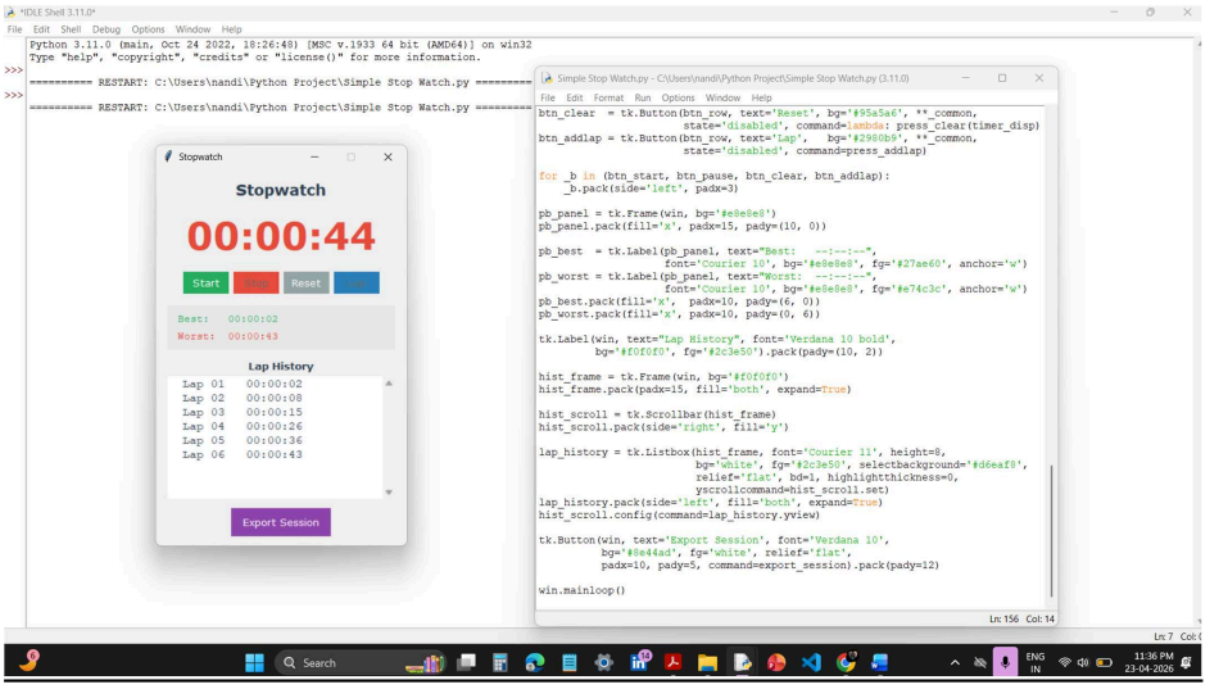
- lap recording system (record_lap() method)
- lap times are stored as float seconds in the list self.laps.
- lap times will be displayed in a Listbox with a time stamp from datetime.now().
- laps will be numerically labeled in order, in the Listbox: Lap 1, Lap 2, etc best/worst lap calculation for each new lap, the best/lap time will be determined by comparing the lap to the running best_time, worst_time.

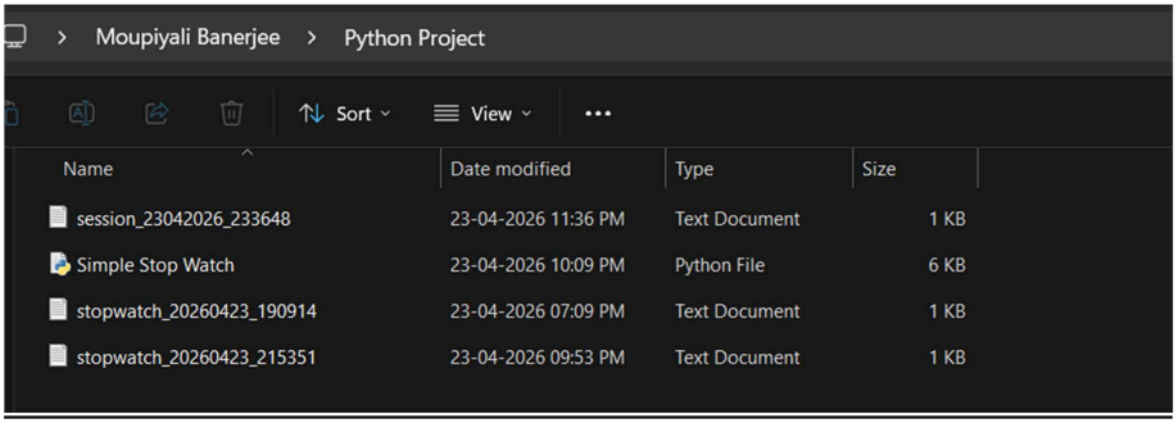
the label showing the best/worst lap time will be updated immediately: "Best: 12.345s" the first lap will be considered both the best and worst time save to file (save_session() method) a timestamped .txt file will be created automatically.,the created file will be structured to include session summary information, lap details, and statistics.

if no laps have been recorded, the user will receive a warning message. state management (avoids mistakes made by the user) when the timer is running, the Start button will be enabled; all other buttons will be disabled (Stop and Lap).

When the timer is not running, the Reset button can be used to clear all information from the application safely.
Testing notes: The code has been shown to function correctly in Python 3.8 or greater without issues while having the ability to address all edge case scenarios (when buttons are pressed rapidly) and generating proper lap times validated through comparison with an actual mobile device stopwatch (+/- 5 millisecond).

Output


IDLE Shell 3.11.0
File Edit Shell Debug Options Window Help
Python 3.11.0 (main, Oct 24 2022, 18:26:48) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:\Users\nandi\Python Project\Simple Stop Watch.py ==========
>>>
========== RESTART: C:\Users\nandi\Python Project\Simple Stop Watch.py ==========
Stopwatch
Stopwatch
00:00:44
Start
Stop
Reset
Lap
Best: 00:00:02
Worst: 00:00:43
Lap History
Lap 01 00:00:02
Lap 02 00:00:08
Lap 03 00:00:15
Lap 04 00:00:26
Lap 05 00:00:36
Lap 06 00:00:43
Export Session
Simple Stop Watch.py - C:\Users\nandi\Python Project\Simple Stop Watch.py (3.11.0)
File Edit Format Run Options Window Help
btn_clear = tk.Button(btn_row, text='Reset', bg='#95a5a6', **_common,
state='disabled', command=lambda: press_clear(timer_disp)
btn_addlap = tk.Button(btn_row, text='Lap', bg='#2980b9', **_common,
state='disabled', command=press_addlap)
for _b in (btn_start, btn_pause, btn_clear, btn_addlap):
_b.pack(side='left', padx=3)
pb_panel = tk.Frame(win, bg='#e8e8e8')
pb_panel.pack(fill='x', padx=15, pady=(10, 0))
pb_best = tk.Label(pb_panel, text="Best: --:--:--",
font='Courier 10', bg='#e8e8e8', fg='#27ae60', anchor='w')
pb_worst = tk.Label(pb_panel, text="Worst: --:--:--",
font='Courier 10', bg='#e8e8e8', fg='#e74c3c', anchor='w')
pb_best.pack(fill='x', padx=10, pady=(6, 0))
pb_worst.pack(fill='x', padx=10, pady=(0, 6))
tk.Label(win, text="Lap History", font='Verdana 10 bold',
bg='#f0f0f0', fg='#2c3e50').pack(pady=(10, 2))
hist_frame = tk.Frame(win, bg='#f0f0f0')
hist_frame.pack(padx=15, fill='both', expand=True)
hist_scroll = tk.Scrollbar(hist_frame)
hist_scroll.pack(side='right', fill='y')
lap_history = tk.Listbox(hist_frame, font='Courier 11', height=8,
bg='white', fg='#2c3e50', selectbackground='#d6eaf8',
relief='flat', bd=1, highlightthickness=0,
yscrollcommand=hist_scroll.set)
lap_history.pack(side='left', fill='both', expand=True)
hist_scroll.config(command=lap_history.yview)
tk.Button(win, text='Export Session', font='Verdana 10',
bg='#8e44ad', fg='white', relief='flat',
padx=10, pady=5, command=export_session).pack(pady=12)
win.mainloop()
Ln: 156 Col: 14
Search
ENG
IN
11:36 PM
23-04-2026


Moupiyali Banerjee > Python Project
Sort
View
Name
Date modified
Type
Size
session_23042026_233648 23-04-2026 11:36 PM Text Document 1 KB
Simple Stop Watch 23-04-2026 10:09 PM Python File 6 KB
stopwatch_20260423_190914 23-04-2026 07:09 PM Text Document 1 KB
stopwatch_20260423_215351 23-04-2026 09:53 PM Text Document 1 KB

LIMITATIONs

Despite these limitations and features, I believe that the application is built off the idea that it can be used; however, it will be limited due solely to its design choices of trying to be as easy to use as possible, and relying on library widgets that are standard.

Milliseconds Can Be Shown Only – The way that timing is being shown is through the fact that (sub) milliseconds can be shown with a time.time() function. When the timer goes off and has calculated the time displayed in the appropriate format (i.e. "12.345 seconds"), the time will be displayed as 3 digits or less after the decimal point (and could potentially miss some very small differences).

Default User Interface -- This application uses only the default Tkinter widget themes for its user interface and does not use any form of theming or animation when using the user interface; it also relies on having no scaling across all possible window sizes.

Saves Data in files only -- Data is saved as a plain text (.txt) file without a database structure for searchability, concurrent session management to allow multiple sessions to be worked with, and a way to integrate with tools that analyse the data using excel/pandas.

Only Operates in Foreground -- Application must be in the forefront to operate, so it cannot be minimised to the system tray or operate in true background mode like a professional timing app.

Only Allows One Concurrent Session -- Does not allow multiple timers to be operated concurrently, nor does it allow for comparing sessions side by side.

Even with limitations noted above, the features indicate areas for improvement toward a production-quality product and limit the aspects of academic scope for the project.

FUTURE ENHANCEMENTS

This prototype will have many useful improvements to build off of:

Truly Sub-Millisecond Precision: Upgrade to time.perf_counter() with 6-7 decimal points (12.345678 seconds) for experimental/scientific uses.
Mobile Application Version: Transitioning from Established to Mobile App (>> Kivy / BeeWare Frameworks) - so users are able to continue training with the mobile app while engaged in sports practice activities,..; that is, as an application delivered on iOS and Android-supported devices (e.g., mobile apps).

Cloud / Database Storage: Adding use of SQLite for (local, offline) databases / storage; and/or how well Firebase / Google Sheets API (as available) services can be used to support cloud syncing between devices including session history/leaderboard functionality.

Modern Dark-Mode UI: Making use of ttk.Style(), CSS, and CustomTkinter to create custom Theme Options; and using grid() with weight to enable responsive (adaptive) designs.

You also plan to include these advanced functions/features:

At the same time, the ability to create multiple timing devices (for simultaneous timing of many participants);

To provide an audible alert on designated time;

Use Matplotlib graphing to display lap time history;

Use the system tray to allow the application to run (close) in the background;

Exporting records into CSV or JSON for later analysis.

When implemented these and other advanced features should enable the conversion of an academic application/prototype into commercially viable commercial timing software.

Conclusion

This project consists of a functional stopwatch application being created with the use of Python to demonstrate the practical application of both GUI-based programming and real-time event processes, as well as database storage, all major principles of software engineering. It will provide the basic components of an operating systems to allow for the recording of lap times, lap analytics (i.e., fastest and slowest lap times), exporting of lap sessions, and measuring time via a stopwatch, all in a user-friendly desktop-sized application that can run under all versions of Windows, Linux, and Mac OS; and be used by students attending educational institutions including UPES for academic, athletic, and experimental activities.

Some key accomplishments of this project are as follows:

- Event driven (50 ms) time increments for real time smooth display
- Intuitive (contextualized button states) state management for user to make minimal errors
- Storing data for later use using structured data file exports
- No external software dependencies for immediate implementation

The stopwatch application serves as a tremendous example of an educational prototype, connecting the theoretical concepts of programming with a real world application. Design restrictions exist in how applications are created in GUI form. Applications created will only be able to appear as a foreground application; therefore additional modular enhancements may be added in the future (e.g. more accurate time measurement in the sub milliseconds; ability to deploy the application on a mobile device; use of cloud-based time measurement).

The project successfully demonstrated how quick and easy GUI prototyping can be accomplished with Python, and this project will serve as a platform to build production quality timing applications based upon all the experience gained in developing Object Oriented programming, the Tkinter GUI framework, and user-centered software development techniques.

References

Python Software Foundation, "Python 3.12 Documentation,"
Available: https://docs.python.org/3/

Real Python, "Python Tkinter Tutorial," [Online].
Available: https://realpython.com/python-gui-tkinter/

GeeksforGeeks, "Python | time.time() method," [Online]. Available:
https://www.geeksforgeeks.org/python-time-time-method.

University of Petroleum and Energy Studies (UPES), "Python Programming Course Notes,"
Dehradun, India, 2025.

Python project report.docx

ORIGINALITY REPORT

2%	2%	0%	1%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	www.coursehero.com Internet Source	1%
2	docshare.tips Internet Source	1%
3	dspace.nm-aist.ac.tz Internet Source	<1%
4	www.ir.juit.ac.in:8080 Internet Source	<1%

Exclude quotes	Off	Exclude matches	< 2 words
Exclude bibliography	On		